

Sales Performance Analysis

SQL Documentation

Database Creation

Data Cleaning

Constraints

Basic Queries

Aggregate Queries

CASE Statements

Views

Subqueries

Stored Procedures

Functions

Window Functions

Prepared by;

Nouf Alhusayyan

Introduction

This document presents the SQL implementation for the Sales Performance Analysis project. It includes the database preparation process, data cleaning, query development, and advanced SQL techniques used before building the Power BI dashboard.

1. Insert Data

```
1 • use sales_performance_analysis;  
2 • show tables;
```

2. Display Data

```
1 • use sales_performance_analysis;  
2 • select * from customers  
3   limit 5;
```

3. Check Data Types

```
1 • use sales_performance_analysis;  
2 • describe sales;
```

4. Convert Date

```
1 • use sales_performance_analysis;  
2 • SELECT  
3     Date,  
4     STR_TO_DATE(Date, '%m/%d/%Y') AS New_Date  
5   FROM sales  
6   LIMIT 10;
```

5. Modify Column Type

```
1 • use sales_performance_analysis;  
2 • alter table sales  
3   modify column Date date;
```

6. Rename Columns

```
1 • use sales_performance_analysis;
2 • alter table sales
3   rename column `id` to sale_id;
```

7. Find Duplicates

```
1 • use sales_performance_analysis;
2 • select product_id , count(*) from products
3   group by product_id
4   having count(*) > 1;
```

8. Check NULL Values

```
1 • use sales_performance_analysis;
2 • select * from products
3   where product_id is null;
```

9. Primary Keys

```
1 • use sales_performance_analysis;
2 • alter table sales
3   ADD primary key (Sale_id);
-
```

10. Foreign Keys

```
1 • use sales_performance_analysis;
2 • alter table products
3   add constraint fk_supplier
4   foreign key (supplier_id)
5   references supplier(supplier_id);
```

11. Basic Queries

```
1 • use sales_performance_analysis;
2 • select count(*) from sales;
```

12. Aggregate Queries

```
1 • use sales_performance_analysis;  
2 • select c.name , sum(s.price) as total_price from customers c  
3   join sales s  
4   on c.customer_id = s.customer_id  
5   group by c.name  
6   order by total_price desc  
7   limit 5;
```

name	total_price
Quinn Hall	4240.5
Alice King	4189.5
Rachel Hill	4177.5
Henry Parker	4173
Isla Walker	4013.5

13. CASE Statements

```
1 • use sales_performance_analysis;  
2 • select p.product_name , s.discount ,  
3   case  
4     when s.discount >= 0.05 then 'High discount'  
5     when s.discount >= 0.03 then 'Medium discount'  
6     else 'Low Discount'  
7   end as Discount_Level  
8   from products p  
9   join sales s  
10  on p.product_id = s.product_id;
```

	product_name	discount	Discount_Level
	Tool C	0.054511332	High discount
	Tool C	0.045399168	Medium discount
	Tool C	0.048945264	Medium discount
	Tool C	0.029699384	Low Discount
	Tool C	0.093234211	High discount

14. Views

```
1 • use sales_performance_analysis;  
2 • create view Customer_Sales as  
3   select c.name , sum(s.price) as total_sales from customers c  
4   join sales s  
5   on c.customer_id = s.customer_id  
6   group by c.name ;
```

```
1 • use sales_performance_analysis;  
2 • select * from customer_sales  
3   order by total_sales desc  
4   limit 5;
```

name	total_sales
Quinn Hall	4240.5
Alice King	4189.5
Rachel Hill	4177.5
Henry Parker	4173
Isla Walker	4013.5

15. Subqueries

```
1 • use sales_performance_analysis;
2 • SELECT p.product_name, s.price FROM products p
3 JOIN sales s
4 ON p.product_id = s.product_id
5 WHERE s.price > ( SELECT AVG(price) FROM sales );
```

product_name	price
Gadget AF	48
Gadget V	54
Widget AL	39
Widget AL	39
Device AI	63

16. Stored Procedure

```
1 • use sales_performance_analysis;
2  Delimiter $$
3 • create procedure CustomerSales (
4   in min_sales decimal (10,2))
5   begin
6   select c.name , sum(s.price) as total_sales from customers c
7   join sales s
8   on c.customer_id = s.customer_id
9   group by c.name
10  having sum(s.price) >= min_sales;
11  end $$
12  delimiter ;
```

name	total_sales
Henry Parker	4173
Alice King	4189.5
Quinn Hall	4240.5
Rachel Hill	4177.5
Isla Walker	4013.5

```
1 • use sales_performance_analysis;
2  Delimiter $$
3 • call CustomerSales (4000) ;
```

17. Function

```
1 • use sales_performance_analysis;
2   DELIMITER $$
3 • CREATE FUNCTION AveragePrice()
4   RETURNS DECIMAL(10,2)
5   DETERMINISTIC
6   BEGIN
7   DECLARE avg_price DECIMAL(10,2);
8   SELECT AVG(price) INTO avg_price FROM sales;
9   RETURN avg_price;
10  END $$
11  DELIMITER ;
```

AveragePrice()
38.23

```
1 • use sales_performance_analysis;
2 • select AveragePrice();
```

18. Window Functions

```
1 • use sales_performance_analysis;  
2 • select rank() over (order by sum(s.price) desc) as customer_Rank ,  
3   c.name , sum(s.price) as total_sales  
4   from customers c  
5  join sales s  
6  on c.customer_id = s.customer_id  
7  group by c.name  
8  order by total_sales desc ;
```

customer_Rank	name	total_sales
1	Quinn Hall	4240.5
2	Alice King	4189.5
3	Rachel Hill	4177.5
4	Henry Parker	4173
5	Isla Walker	4013.5

Conclusion

This project demonstrates the practical application of SQL for data preparation and business analysis. The implemented queries include data cleaning, joins, aggregate functions, CASE statements, views, subqueries, stored procedures, user-defined functions, and window functions. The processed data was then used to build an interactive Power BI dashboard for sales performance analysis.
